VendoRL: Adaptive Pricing for a Vending Machine

A Student Mini-Project Using Reinforcement Learning

Shiv JEE Yadav (B23EE1095) **Ritik Nagar (B23EE1086)**
IIT Jodhpur
GitHub: https://github.com/ritiknagar9521/VendoRL

1 Introduction

Imagine running a vending machine for a full day. If prices are too low, products may sell quickly but the profit on each sale is small. If prices are too high, fewer customers may buy anything, leaving products unsold. Choosing the right price becomes even harder when customer demand changes throughout the day.

VendoRL is a project that explores how a vending machine can learn better prices using reinforcement learning. We created a simulated machine selling snacks and drinks and trained an agent using Proximal Policy Optimization (PPO).

The main aim is to earn higher overall profit by adjusting prices according to available stock, recent sales, time of day, and competition.

2 Problem and Environment

Our simulated vending machine operates for a 12-hour shift, starting at 08:00. At the beginning of the shift, it has 30 snacks and 30 drinks. Every hour, the agent chooses prices, customers make random purchases, and the remaining stock is updated.

The main environment settings are:

- Snack prices: Rs 20, Rs 40, and Rs 70.
- Drink prices: Rs 30, Rs 50, and Rs 90.
- Purchase cost: Rs 15 per snack and Rs 20 per drink.
- Holding cost: Rs 1 per remaining unit per hour.
- Lunch rush: 12:00–14:00.
- Competitor cafeteria: open from 12:00–16:00.

Customer demand is random and depends on price, time, and the demand level of the day. We consider three possible demand levels: low, normal, and high, with probabilities of 30%, 40%, and 30%. The environment uses a Poisson distribution to generate customer purchases, and sales cannot exceed the available stock.

An important detail is that the agent never directly sees the day's demand category. It must use recent sales and other observations to make reasonable pricing decisions.

3 Reinforcement Learning Formulation

We model the problem as a Markov Decision Process (MDP), where the agent observes the current situation, chooses an action, and receives a reward.

State Space (S):

The state space is represented as an 8-dimensional continuous vector, $S \in [0,1]^8$. At each timestep t, the state s_t is defined as:

$$s_t = (I_s, I_d, \tau, C, q_{s,t-1}, q_{d,t-1}, p_{s,t-1}, p_{d,t-1})$$

where I_s, I_d are the remaining normalized snack and drink inventories, τ is the normalized time in the shift, $C \in \{0,1\}$ indicates if the competitor is open, q represents normalized previous sales, and p represents previous price tiers. The hidden demand category is intentionally excluded.

Action Space (A):

The action space is discrete, defined as $A = \{0,1,\ldots,8\}$. Each action $a_t \in A$ maps directly to a specific price combination tuple (P_s, P_d) where $P_s \in \{20,40,70\}$ and $P_d \in \{30,50,90\}$. This allows independent price adjustment for both products.

Reward Function (R):

The reward function R_t maximizes profit while penalizing excess inventory holding. For a given timestep t:

$$R_t = (P_{s,t} - c_s)q_{s,t} + (P_{d,t} - c_d)q_{d,t} - h(I_{s,t} + I_{d,t})$$

where c_s, c_d are the base purchase costs, $q_{s,t}, q_{d,t}$ are the actual units sold, and $h = 1$ is the holding cost per unit.

At the terminal step T (end of the shift), a final penalty is applied for the purchase cost of all unsold inventory:

$$R_{T(\text{final})} = R_T - (c_s I_{s,T} + c_d I_{d,T})$$

This formulation ensures the agent balances margin maximization with efficient inventory liquidation.

4 Why PPO?

We selected Proximal Policy Optimization (PPO) because it is a widely used reinforcement learning algorithm that learns a policy through repeated interaction with an environment.

Our implementation uses two neural networks:

- **Actor:** Selects among the nine pricing actions.
- **Critic:** Estimates the expected future reward and helps the actor judge its decisions.

Both networks have two hidden layers containing 64 units each, with Tanh activation functions. During training, the agent tries different prices, observes the resulting rewards, and updates its policy. PPO limits the size of policy updates to help keep learning stable.

The main training settings are a learning rate of 0.0003, a discount factor of 0.99, eight update epochs per batch, and 32 episodes collected per training iteration. The default training script uses 400 iterations for each of three random seeds.

5 Baseline Strategies

We compare PPO with three simple strategies to understand whether learning improves the pricing decisions.

- **Random pricing:** Selects a pricing action randomly.
- **Fixed pricing:** Always charges Rs 40 for a snack and Rs 50 for a drink.
- **Time-based rule:** Uses cheaper prices while the competitor is open and normal prices at other times.

The fixed-price strategy provides a useful reference because it does not adapt to changing demand. PPO, on the other hand, can change its decisions as the shift progresses.

6 Results

The dashboard below summarizes the training curve, profits, unsold stock, hourly prices, and profit under different demand levels.

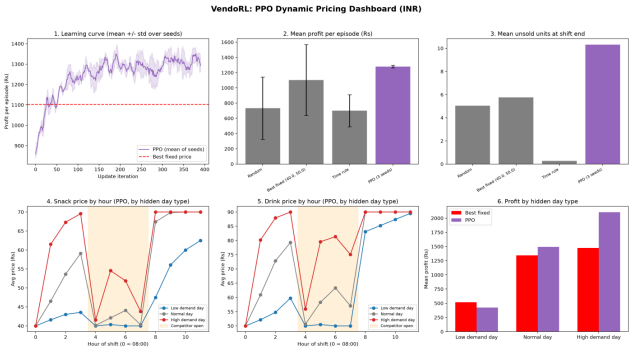


Figure 1: VendoRL results dashboard showing training progress, strategy comparisons, inventory, hourly prices, and profit across demand levels.

Approximate readings from the supplied dashboard are shown below.

Strategy	Profit (Rs)	Unsold units
Random	730	5.0
Time-based	700	0.3
Fixed price	1100	5.8
PPO	**1280**	**10.3**

These figures are approximate readings from the dashboard and should be checked against the final evaluation output before submission.

The dashboard suggests that PPO earns more average profit than the baseline strategies. Its displayed average is approximately Rs 1,280 per shift, compared with Rs 1,100 for fixed pricing. This represents an improvement of roughly 16% based on these values.

The results also show a trade-off. PPO leaves more products unsold on average, and its performance is weaker on low-demand days. Its advantage appears larger on high-demand days, where adjusting prices can help the machine earn more from customer purchases.

7 Conclusion

VendoRL demonstrates how reinforcement learning can be used to solve a familiar pricing problem. PPO learns from simulated experience and adjusts prices according to stock, time, competition, and recent sales.

The supplied dashboard suggests that PPO can improve average profit compared with simple pricing strategies, particularly under higher demand. At the same time, the larger amount of unsold stock shows that higher profit does not always mean better inventory management. Testing the approach with real sales data would be a useful next step.

References

- J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.
- J. Schulman et al., "High-Dimensional Continuous Control Using Generalized Advantage Estimation," ICLR, 2016.
- R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.